```
const fs = require("fs");
const glob = require("glob");
const { execSync } = require("child_process");

function sortImports(content) {
  const lines = content.split("\n");

  let useClient = [];
  let imports = [];
  let others = [];

  let currentImportBlock = "";
  let isCollectingImport = false;

  for (let i = 0; i < lines.length; i++) {
    const line = lines[i];
    const trimmed = line.trim();

    if (
      trimmed.startsWith('"use client"') ||
      trimmed.startsWith("'use client'")
    ) {
      useClient.push(line);
      continue;
    }

    if (trimmed.startsWith("import ") || isCollectingImport) {
      currentImportBlock += (currentImportBlock ? "\n" : "") + line;

      const hasOpeningBrace = trimmed.includes("{");
      const hasClosingBrace = trimmed.includes("}");

      if (hasOpeningBrace && !hasClosingBrace) {
        isCollectingImport = true;
      } else if (hasClosingBrace) {
        isCollectingImport = false;
      }

      if (!isCollectingImport) {
        if (currentImportBlock.includes("import")) {
          imports.push(currentImportBlock);
          currentImportBlock = "";
        } else {
          others.push(currentImportBlock);
          currentImportBlock = "";
        }
      }
      continue;
    }

    others.push(line);
  }

  imports.sort((a, b) => a.length - b.length);

  let result = "";

  if (useClient.length > 0) {
    result += useClient.join("\n") + "\n";
  }

  if (imports.length > 0) {
    result += imports.join("\n");
  }
```

```
  const codeContent = others.join("\n").trim();

  if (codeContent) {
    if (imports.length > 0 || useClient.length > 0) {
      result += "\n\n" + codeContent;
    } else {
      result += codeContent;
    }
  }

  return result + "\n";
}

function sortCssContent(content) {
  const lines = content.split("\n");
  let sortedContent = [];
  let stack = [];

  function flushBlock(block) {
    if (block.lines.length > 0) {
      block.lines.sort((a, b) => a.trim().length - b.trim().length);
      block.sortedResult.push(...block.lines);
      block.lines = [];
    }
  }

  lines.forEach((line) => {
    const trimmed = line.trim();
    if (trimmed.endsWith("{")) {
      stack.push({ sortedResult: [line], lines: [] });
    } else if (trimmed === "}") {
      const finishedBlock = stack.pop();
      if (finishedBlock) {
        flushBlock(finishedBlock);
        finishedBlock.sortedResult.push(line);
        if (stack.length > 0) {
          stack[stack.length - 1].sortedResult.push(
            ...finishedBlock.sortedResult,
          );
        } else {
          sortedContent.push(...finishedBlock.sortedResult);
        }
      }
    } else {
      if (stack.length > 0) {
        if (
          trimmed.includes(":") &&
          (trimmed.endsWith(";") || trimmed.endsWith("!important"))
        ) {
          stack[stack.length - 1].lines.push(line);
        } else {
          stack[stack.length - 1].sortedResult.push(line);
        }
      } else {
        sortedContent.push(line);
      }
    }
  });

  return sortedContent.join("\n");
}

function sortProject() {
```

```
  console.log("🚀 Starting script...");

  const cssFiles = glob.sync("src/**/*.module.css");
  cssFiles.forEach((file) => {
    const content = fs.readFileSync(file, "utf-8");
    fs.writeFileSync(file, sortCssContent(content), "utf-8");
    console.log(`✅ Sorted CSS: ${file}`);
  });

  const codeFiles = glob.sync("src/**/*.{js,jsx,ts,tsx}");
  codeFiles.forEach((file) => {
    const content = fs.readFileSync(file, "utf-8");
    fs.writeFileSync(file, sortImports(content), "utf-8");
    console.log(`✅ Sorted Imports: ${file}`);
  });

  try {
    console.log("✨ Running Prettier...");
    execSync("npx prettier --write . --log-level warn", { stdio: "inherit" });
    console.log("🎉 Success!");
  } catch (err) {
    console.error("⚠️ Prettier failed.");
  }
}

if (require.main === module) {
  sortProject();
}

module.exports = { sortProject };
```